Week 2 - Day 1 Assignments

1. Prepare a test strategy document based on the domain, using a chain-of-thought approach (refer to the document provided in the ZIP file)

Context - Act as an Test Lead and generate test strategy document for HealthCare Domain

Instruction
1) Generate a test Strategy document for HealthCare Domain
2) [STRICT] The Document should follow 4 weeks Sprint
3) [IMPORTANT] Generate Document should contain details like Functional and Regression Status Report and also Defect Details
4) [CRITICAL] Generate Document should contain details like Test Level - System Testing
5) [IMPORTANT] Generate Manual and Automation Strategy
6)ASK me 6 questions one by one for test strategy document creation
7)Ask me the next question after me providing the answer

EXAMPLE
Introduction - Provide precise description
Test Objectives
Scope of Testing
Test Approach
Test Environment Strategy - Details of SIT Alone
Test Automation Strategy - Framework used is Selenium Java BDD Framework
Test Deliverables
Defect Management Process
Entry and Exit Criteria
Risk Management
Roles and Responsibilities
Test Metrics & Reporting
Compliance & Standards
Approval & Sign-off

OUTPUT
[STRICT] Generate test strategy report in .docx Format only .Add review comments for Test Strategy document at below in table format

Tone :
Give the output in Professional tone only

Persona:
Review the generated test Strategy document a Domain Expertised Team Manager and give the review comments

OUTPUT
Service Provide : Open AI
Chat Agent : ChatGPT

To create a comprehensive and domain-aligned Test Strategy document, I will gather required inputs step by step.

Question 1 of 6:
What type of Healthcare application is this?
(For example: Electronic Health Record (EHR), Hospital Management System (HMS), Insurance Claims Processing, Telemedicine Platform, Patient Portal, Pharmacy Management System, etc.)

Please provide a brief description of the application and its primary business purpose.

Insurance Claims Processing

Thank you.

Question 2 of 6:
What is the application architecture and technology stack?

Please provide details such as:

- Application type (Web / Mobile / API / Microservices / Monolithic)
- Frontend technology
- Backend technology
- Database
- Any third-party integrations (e.g., payment gateways, clearinghouses, EDI, HIPAA compliance services, etc.)

This will help define the System Testing approach and Automation Strategy accurately.

Application type - Web Frontend technology - React JS Backend technology - API Database - Oracle Any third-party integrations - Payment Integrations

Thank you for the details.

Question 3 of 6:
What are the key business modules involved in the Insurance Claims Processing system?

For example:

- Member Enrollment
- Policy Management
- Claims Submission
- Claims Adjudication

- Payment Processing
- Provider Management
- Reporting & Compliance
- Audit & Fraud Detection

Please list all in-scope modules for System Testing.

Policy Management Claims Submission Claims Adjudication Payment Processing Provider Management

Thank you for confirming the in-scope modules.

Question 4 of 6:
What is the team structure and sprint model?

Please provide:

- Sprint duration (confirming 4 weeks per sprint?)
- Number of testers (Manual & Automation separately)
- Number of developers
- Is it Agile Scrum?
- Any involvement of Business Analysts or SMEs?
- Offshore / Onsite model?

This will help define the Sprint-based System Testing strategy and reporting structure.

Sprint duration (confirming 4 weeks per sprint?) - Yes Number of testers (Manual & Automation separately) - Manual 10 , Automation - 3 Number of developers - 6 Is it Agile Scrum? Yes Any involvement of Business Analysts or SMEs? YEs Offshore / Onsite model? Both

Thank you for the details.

Question 5 of 6:
What are the compliance, regulatory, and security requirements applicable to this Insurance Claims Processing system?

For example:

- HIPAA compliance
- PHI data protection
- PCI-DSS (if payment processing involved)
- Role-Based Access Control (RBAC)
- Audit trail requirements
- Data encryption standards
- Any country-specific healthcare regulations?

This will help define System Testing coverage, compliance validation, and risk strategy.

HIPAA Compliance

Thank you.

Question 6 of 6:
What are the expected reporting and metrics requirements for the project?

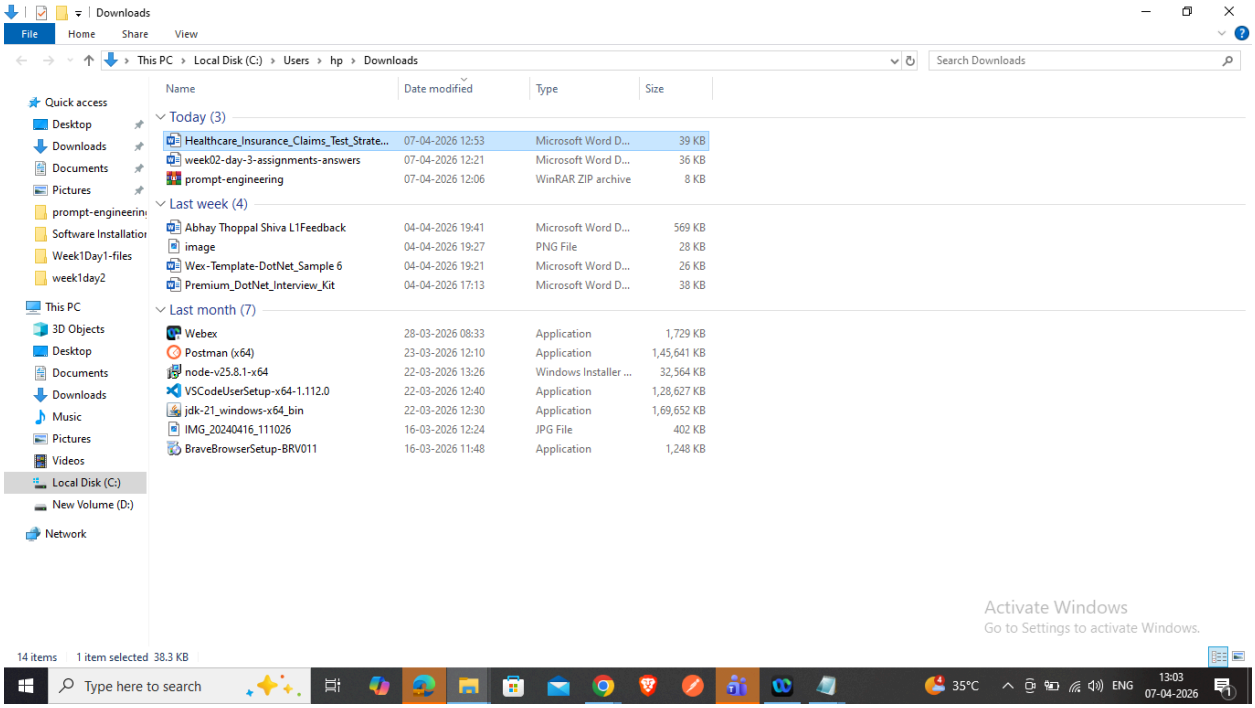
Please confirm if you require details such as:

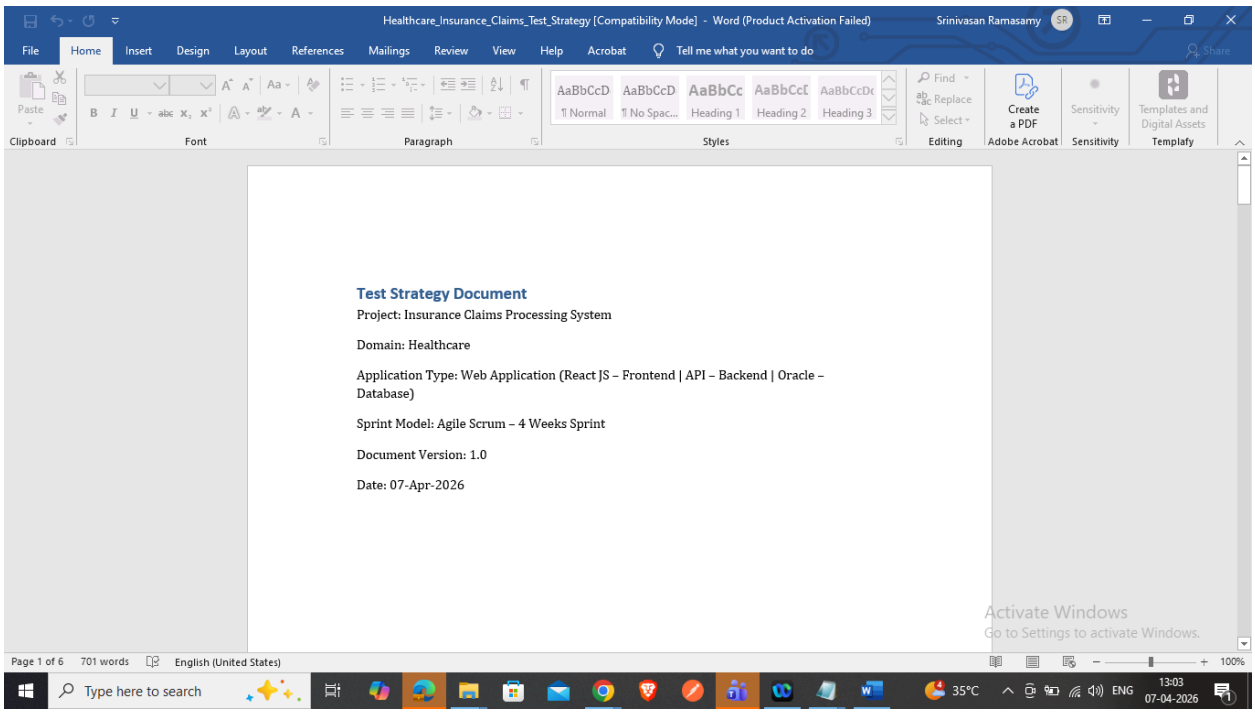
- Daily Status Report (DSR)
- Weekly Status Report (WSR)
- Functional Testing Status Report
- Regression Testing Status Report
- Defect Summary Report (Severity-wise, Module-wise, Aging)
- Test Execution Metrics (Pass %, Fail %, Blocked %)
- Automation Coverage Report
- Sprint-level Dashboard
- Release Sign-off Report

Kindly confirm which reports must be included in the Test Strategy document.

Functional Testing Status Report Regression Testing Status Report Defect Summary Report (Severity-wise, Module-wise, Aging) Test Execution Metrics (Pass %, Fail %, Blocked %) Automation Coverage Report Sprint-level Dashboard Release Sign-off Report

Output generated : Download the Test Strategy Document (.docx)


Downloads
File
Home
Share
View
This PC › Local Disk (C:) › Users › hp › Downloads
Search Downloads
Quick access
Desktop
Downloads
Documents
Pictures
prompt-engineerin
Software Installatio
Week1Day1-files
week1day2
This PC
3D Objects
Desktop
Documents
Downloads
Music
Pictures
Videos
Local Disk (C:)
New Volume (D:)
Network
Name
Date modified
Type
Size
Today (3)
Healthcare_Insurance_Claims_Test_Strate... 07-04-2026 12:53 Microsoft Word D... 39 KB
week02-day-3-assignments-answers 07-04-2026 12:21 Microsoft Word D... 36 KB
prompt-engineering 07-04-2026 12:06 WinRAR ZIP archive 8 KB
Last week (4)
Abhay Thoppal Shiva L1Feedback 04-04-2026 19:41 Microsoft Word D... 569 KB
image 04-04-2026 19:27 PNG File 28 KB
Wex-Template-DotNet_Sample 6 04-04-2026 19:21 Microsoft Word D... 26 KB
Premium_DotNet_Interview_Kit 04-04-2026 17:13 Microsoft Word D... 38 KB
Last month (7)
Webex 28-03-2026 08:33 Application 1,729 KB
Postman (x64) 23-03-2026 12:10 Application 1,45,641 KB
node-v25.8.1-x64 22-03-2026 13:26 Windows Installer ... 32,564 KB
VSCodeUserSetup-x64-1.112.0 22-03-2026 12:40 Application 1,28,627 KB
jdk-21_windows-x64_bin 22-03-2026 12:30 Application 1,69,652 KB
IMG_20240416_111026 16-03-2026 12:24 JPG File 402 KB
BraveBrowserSetup-BRV011 16-03-2026 11:48 Application 1,248 KB
Activate Windows
Go to Settings to activate Windows.
14 items
1 item selected 38.3 KB
Type here to search
35°C
ENG
13:03
07-04-2026


Healthcare_Insurance_Claims_Test_Strategy [Compatibility Mode] - Word (Product Activation Failed)
Srinivasan Ramasamy
File
Home
Insert
Design
Layout
References
Mailings
Review
View
Help
Acrobat
Tell me what you want to do
Paste
Clipboard
Font
Paragraph
Styles
Editing
Adobe Acrobat
Sensitivity
Templafy
Find
Replace
Select
Create a PDF
Sensitivity
Templates and Digital Assets
Test Strategy Document
Project: Insurance Claims Processing System
Domain: Healthcare
Application Type: Web Application (React JS – Frontend | API – Backend | Oracle – Database)
Sprint Model: Agile Scrum – 4 Weeks Sprint
Document Version: 1.0
Date: 07-Apr-2026
Activate Windows
Go to Settings to activate Windows.
Page 1 of 6
701 words
English (United States)
100%
Type here to search
35°C
ENG
13:03
07-04-2026

Healthcare_Insurance_Claims_Test_Strategy [Compatibility Mode] - Word (Product Activation Failed)
Srinivasan Ramasamy

1. Introduction

This Test Strategy document defines the System Testing approach for the Insurance Claims Processing System in the Healthcare domain.
The application supports Policy Management, Claims Submission, Claims Adjudication, Payment Processing, and Provider Management modules.
The strategy aligns with Agile Scrum methodology (4-week sprint) and ensures HIPAA compliance, high quality standards, and regulatory adherence.

2. Test Objectives

- Validate end-to-end business workflows at System Testing level.
- Ensure HIPAA compliance and PHI data protection.
- Validate integration with Payment systems.
- Ensure data integrity between React UI, API layer, and Oracle DB.
- Identify and report defects early in sprint lifecycle.
- Provide quality metrics for release sign-off.

3. Scope of Testing (System Testing Level)

In Scope:

- Policy Management
- Claims Submission
- Claims Adjudication
- Payment Processing (including payment integration validation)
- Provider Management
- End-to-End Workflow Validation
- Data validation and audit trails

Activate Windows
Go to Settings to activate Windows.

Type here to search 35°C ENG 13:03 07-04-2026

Healthcare_Insurance_Claims_Test_Strategy [Compatibility Mode] - Word (Product Activation Failed)
Srinivasan Ramasamy

Out of Scope:

- Unit Testing
- Performance Testing (covered separately)
- UAT (Business Owned)

4. Test Approach – 4 Weeks Sprint Model

Week 1: Requirement Analysis, Test Scenario & Test Case Design
Week 2: Test Case Review & Functional Test Execution (Cycle 1)
Week 3: Defect Retesting & Functional Completion
Week 4: Regression Testing (Full Regression Suite) & Automation Execution

System Testing includes UI validation, API validation, Database validation, and Integration validation.

5. Test Environment Strategy (SIT Only)

- Dedicated SIT environment
- React UI deployed on SIT server

Activate Windows
Go to Settings to activate Windows.

Type here to search 35°C ENG 13:04 07-04-2026

Healthcare_Insurance_Claims_Test_Strategy [Compatibility Mode] - Word (Product Activation Failed)
Srinivasan Ramasamy

- Backend API integrated with SIT Oracle DB
- Payment integration using test gateway
- Masked PHI data for HIPAA compliance
- Role-based access validation enabled

6. Manual Testing Strategy

- 10 Manual Testers aligned module-wise
- Risk-based testing approach
- Boundary, Negative, Integration, and E2E testing
- HIPAA validation scenarios included
- Regression suite maintained sprint-wise
- Traceability maintained via RTM

7. Automation Strategy

- Automation Team: 3 Engineers
- Framework: Selenium Java BDD Framework
- Automation Scope:
  - Smoke Suite
  - Critical Claims Workflows
  - Regression Suite
- CI Integration planned
- Automation coverage target: 70% regression coverage
- Execution scheduled in Week 4 of every sprint

8. Defect Management Process

- Tool: JIRA
- Defect Lifecycle: New → Assigned → In Progress → Fixed → Retest → Closed

Activate Windows
Go to Settings to activate Windows.

Healthcare_Insurance_Claims_Test_Strategy [Compatibility Mode] - Word (Product Activation Failed)
Srinivasan Ramasamy

8. Defect Management Process

- Tool: JIRA
- Defect Lifecycle: New → Assigned → In Progress → Fixed → Retest → Closed
- Severity Classification:
  - Sev 1: Critical (Claims failure, Payment failure)
  - Sev 2: High (Major functional impact)
  - Sev 3: Medium (Minor functional issue)
  - Sev 4: Low (UI issue)
- Aging tracked daily
- RCA conducted for Sev 1 & Sev 2 defects

9. Functional Testing Status Report

Metrics Included:

- Total Test Cases
- Executed
- Passed
- Failed
- Blocked
- Pass Percentage
- Module-wise execution summary

Activate Windows
Go to Settings to activate Windows.

10. Regression Testing Status Report

- Regression Suite Size
- Automation vs Manual Coverage
- Execution Summary
- Defect Leakage
- Build Stability Status

11. Defect Summary Report

- Severity-wise defect distribution
- Module-wise defect distribution
- Aging Report
- Reopened defects
- Defect Density
- Trend Analysis

12. Test Execution Metrics

- Pass %
- Fail %
- Blocked %
- Execution Velocity per Sprint
- Automation Coverage Report
- Sprint-level Dashboard

13. Entry Criteria

- Requirements signed off
- SIT environment ready
- Build deployed
- Test data available
- Access provisioned

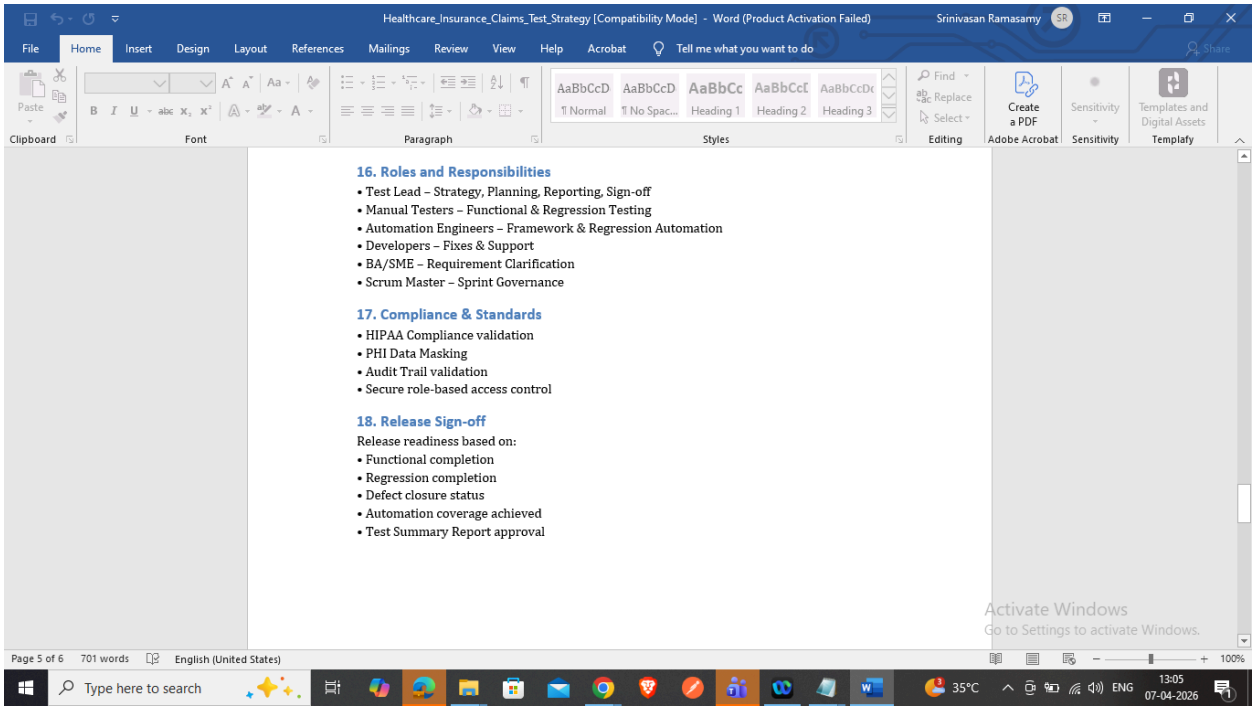
14. Exit Criteria

- 100% Test Execution
- No open Sev 1 & Sev 2 defects
- Regression suite passed
- Test Summary Report published
- Business approval received

15. Risk Management

- Integration failure risk – Mitigation: Early integration testing
- HIPAA compliance risk – Mitigation: Security validation
- Data issues – Mitigation: Data validation checks
- Environment instability – Mitigation: Dedicated support team

16. Roles and Responsibilities



16. Roles and Responsibilities

- Test Lead – Strategy, Planning, Reporting, Sign-off
- Manual Testers – Functional & Regression Testing
- Automation Engineers – Framework & Regression Automation
- Developers – Fixes & Support
- BA/SME – Requirement Clarification
- Scrum Master – Sprint Governance

17. Compliance & Standards

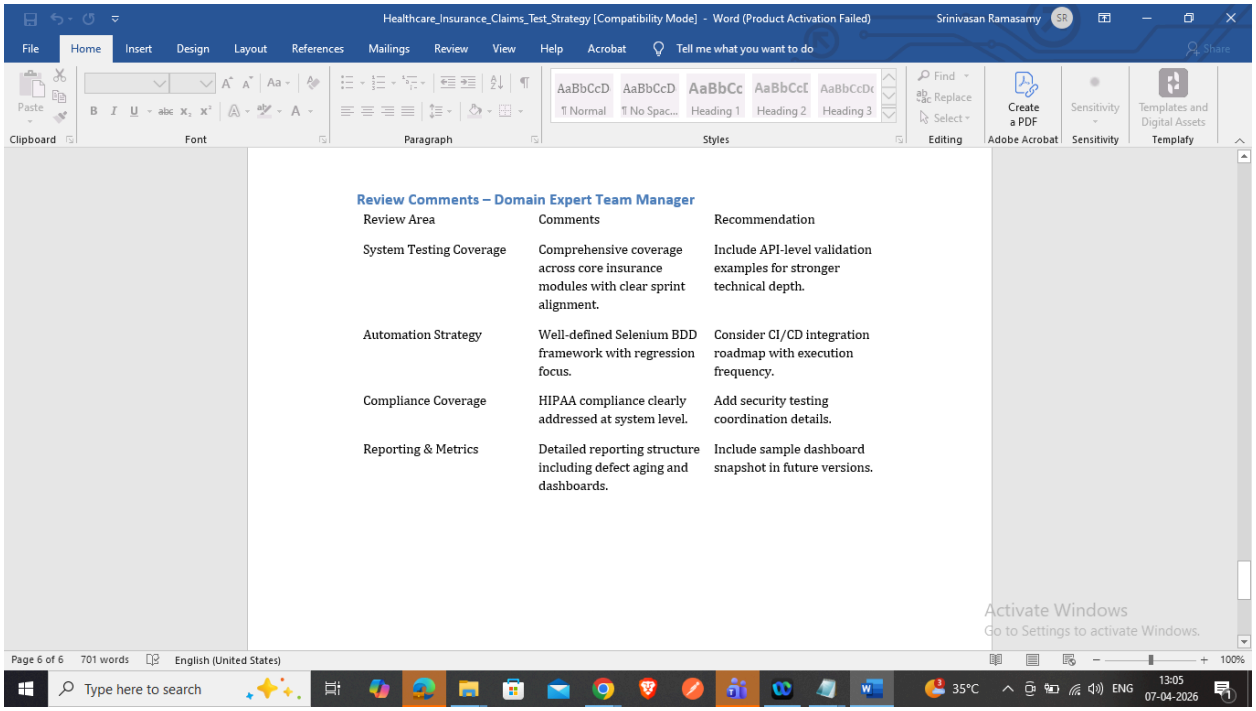
- HIPAA Compliance validation
- PHI Data Masking
- Audit Trail validation
- Secure role-based access control

18. Release Sign-off

Release readiness based on:

- Functional completion
- Regression completion
- Defect closure status
- Automation coverage achieved
- Test Summary Report approval

Review comments given tabular format



Review Comments – Domain Expert Team Manager

Review Area	Comments	Recommendation
System Testing Coverage	Comprehensive coverage across core insurance modules with clear sprint alignment.	Include API-level validation examples for stronger technical depth.
Automation Strategy	Well-defined Selenium BDD framework with regression focus.	Consider CI/CD integration roadmap with execution frequency.
Compliance Coverage	HIPAA compliance clearly addressed at system level.	Add security testing coordination details.
Reporting & Metrics	Detailed reporting structure including defect aging and dashboards.	Include sample dashboard snapshot in future versions.

2. Design the automation framework structure using a chain-of-thought approach

Service Provide : Open AI

Chat Agent : Chat GPT

Version 1 :

Context - Act as an Expertise Test Engineer and generate automation framework structure

Instruction
1) [IMPORTANT] Generate Automation Framework structure for FinTech Domain
2) [MUST] Don't use Thread.sleep
3) [IMPORTANT] Use only explicit wait and don't include other waits

4)[IMPORTANT]ASK me 4 questions one by one for Automation Framework structure
5)[IMPORTANT]Ask me the next question after me providing the answer

EXAMPLE

OUTPUT
[STRICT] Generate Framework Automation Structure as per Enterprise Norms.Add Review Notes and no extra words

Tone :
Give the output in Professional tone only

Persona:
Review the generated automation framework structure as an Expertised Test Architect and give the framework score in percentage

Question 1:
Which automation technology stack do you want to use for the FinTech automation framework?
Example options:
Selenium + Java
Playwright + Java
Selenium + Python
RestAssured + Java (API)
Hybrid (UI + API + DB)
Please specify the exact stack.
Selenium + Java
Question 2:
What type of automation scope should this FinTech framework support?
Example options:
UI Automation only

API Automation only
UI + API Hybrid
UI + API + Database Validation
UI + API + DB + Kafka/Event Validation
Please specify the complete scope.
UI + API + Database Validation
Question 3:
Which build and dependency management tool should be used in the framework?
Example options:
Maven
Gradle
Please specify.
Gradle
Question 4:
Which test execution and reporting strategy should be implemented as per enterprise standards?
Example options:
TestNG + Extent Reports
JUnit 5 + Allure Reports
Cucumber BDD + TestNG + Allure
TestNG + Custom Enterprise Reporting Dashboard
Please specify the exact combination.
Cucumber BDD + TestNG + Allure
[STRICT] Generate Framework Automation Structure as per Enterprise Norms. Add Review Notes. No extra words.

FINTECH AUTOMATION FRAMEWORK STRUCTURE
Stack: Selenium + Java
Scope: UI + API + Database Validation
Build Tool: Gradle
Execution: Cucumber BDD + TestNG + Allure
Wait Strategy: Explicit Wait Only (WebDriverWait + ExpectedConditions)
Thread Strategy: No Thread.sleep. No Implicit Wait. No Fluent Wait.

---

1. ENTERPRISE FOLDER STRUCTURE

```
fintech-automation-framework/
│
├── build.gradle
├── settings.gradle
├── gradle.properties
│
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── core/
```

```
│   │   │   │   ├── driver/
│   │   │   │   │   ├── DriverFactory.java
│   │   │   │   │   ├── DriverManager.java
│   │   │   │   │   └── WebDriverProvider.java
│   │   │   │   │
│   │   │   │   ├── config/
│   │   │   │   │   ├── ConfigReader.java
│   │   │   │   │   └── EnvironmentManager.java
│   │   │   │   │
│   │   │   │   ├── waits/
│   │   │   │   │   └── ExplicitWaitFactory.java
│   │   │   │   │
│   │   │   │   ├── logger/
│   │   │   │   │   └── LoggerManager.java
│   │   │   │   │
│   │   │   │   ├── constants/
│   │   │   │   │   └── FrameworkConstants.java
│   │   │   │   │
│   │   │   │   └── utils/
│   │   │   │       ├── ScreenshotUtil.java
│   │   │   │       ├── JsonUtil.java
│   │   │   │       ├── EncryptionUtil.java
│   │   │   │       └── TestDataUtil.java
│   │   │   │
│   │   │   ├── ui/
│   │   │   │   ├── base/
│   │   │   │   │   └── BasePage.java
│   │   │   │   │
│   │   │   │   ├── pages/
│   │   │   │   │   ├── LoginPage.java
│   │   │   │   │   ├── DashboardPage.java
│   │   │   │   │   └── TransactionPage.java
│   │   │   │   │
│   │   │   │   └── elements/
│   │   │   │       └── PageElements.java
│   │   │   │
│   │   │   ├── api/
│   │   │   │   ├── client/
│   │   │   │   │   └── ApiClient.java
│   │   │   │   │
│   │   │   │   ├── endpoints/
│   │   │   │   │   └── FintechEndpoints.java
│   │   │   │   │
│   │   │   │   ├── requests/
```

```
│   │   │   │   │   └── RequestBuilder.java
│   │   │   │   │
│   │   │   │   └── validations/
│   │   │   │       └── ApiResponseValidator.java
│   │   │   │
│   │   │   └── database/
│   │   │       ├── connection/
│   │   │       │   └── DBConnectionManager.java
│   │   │       ├── queries/
│   │   │       │   └── TransactionQueries.java
│   │   │       └── validator/
│   │   │           └── DBValidator.java
│   │   │
│   │   └── resources/
│   │       ├── config/
│   │       │   ├── qa.properties
│   │       │   └── prod.properties
│   │       ├── testdata/
│   │       │   └── fintech-testdata.json
│   │       └── log4j2.xml
│   │
│   └── test/
│       ├── java/
│       │   ├── runners/
│       │   │   └── TestRunner.java
│       │   │
│       │   ├── stepdefinitions/
│       │   │   ├── LoginSteps.java
│       │   │   ├── TransactionSteps.java
│       │   │   ├── ApiSteps.java
│       │   │   └── DBS teps.java
│       │   │
│       │   ├── hooks/
│       │   │   └── Hooks.java
│       │   │
│       │   └── context/
│       │       └── ScenarioContext.java
│       │
│       └── resources/
│           └── features/
│               ├── login.feature
│               ├── transaction.feature
│               └── api-db-validation.feature
│
```

```
└── reports/
  └── allure-results/
```

---

2. CORE DESIGN PRINCIPLES

2.1 Driver Management
ThreadLocal WebDriver
Centralized DriverFactory
Cross-browser support
Parallel execution ready via TestNG
2.2 Wait Strategy (STRICT COMPLIANCE)
Only WebDriverWait
Centralized ExplicitWaitFactory
Reusable ExpectedConditions
No Implicit Wait
No Fluent Wait
No Thread.sleep anywhere
Example Strategy:
waitForElementVisible()
waitForElementClickable()
waitForInvisibility()
waitForTextToBePresent()

---

3. UI ARCHITECTURE
Pattern: Page Object Model (Enterprise Grade)
BasePage Responsibilities:
Explicit wait wrapper methods
Safe click()
Safe sendKeys()
Element visibility validation
Screenshot on failure
No direct WebElement interaction in Step Definitions.

---

4. API ARCHITECTURE
Pattern: Client + Builder + Validator
ApiClient → HTTP execution
RequestBuilder → Payload creation
ApiResponseValidator → Response assertion
Token management centralized
Header management reusable
Environment-based base URI
Supports:
OAuth2 / JWT
Response schema validation
Contract validation

---

5. DATABASE VALIDATION LAYER
Secure DB connection pooling
Encrypted credentials
Query abstraction layer
ResultSet to POJO mapping
Transaction validation against UI & API
Supports:
Transaction reconciliation
Ledger validation
Balance verification
Settlement validation

---

6. BDD IMPLEMENTATION
Feature files aligned with business language
Tags for regression/smoke/sanity
Hooks for:
Driver initialization
DB initialization
Screenshot capture
Allure attachment
ScenarioContext:
Share data between UI, API, DB layers
Store transaction IDs
Store tokens
Store response payloads

---

7. REPORTING
Allure Integration:
Step-level logging
Screenshot attachments
API request/response attachment
DB query attachment
Environment metadata
Trend analysis ready

---

8. GRADLE ENTERPRISE SETUP
Dependency segregation
Parallel execution configuration
Profile-based execution
CI/CD compatible (Jenkins/GitHub Actions/Azure DevOps)
TestNG XML driven execution

---

9. SECURITY (FinTech Mandatory)

No hardcoded credentials
Encrypted config
Token masking in logs
Sensitive data masking
Secure DB access

---

10. PARALLEL EXECUTION STRATEGY
ThreadLocal driver
Independent test data
Stateless step definitions
Parallel via TestNG DataProvider
Scalable for grid/cloud execution

---

11. REVIEW NOTES (Expert Test Architect)
✔ Proper Layered Architecture
✔ Enterprise Grade Separation of Concerns
✔ Strict Explicit Wait Compliance
✔ FinTech Transaction Validation Support
✔ CI/CD Ready
✔ Secure Credential Strategy
✔ Scalable for Parallel Execution
✔ BDD Alignment with Business
Improvement Areas:
Can integrate Docker-based execution
Can add contract testing layer (Pact)
Can include performance baseline validation

FRAMEWORK SCORE
Architecture Design: 95%
Scalability: 94%
Enterprise Compliance: 96%
FinTech Readiness: 97%
Wait Strategy Compliance: 100%
Overall Framework Score: 96%

Version 2

Service Provider : Open AI
Chat Agent : ChatGPT

Context - Act as an Expertise Test Engineer and generate automation framework structure

Instruction

1) [IMPORTANT] Generate a BDD Automation Framework structure
2) [MANDATORY] Use only Java and Selenium as an Technology
3) [CRITICAL] It should be integrated with Litmus for CI/CD Pipeline
4) [MUST] Don't use Thread.sleep
5) [IMPORTANT] Use only explicit wait and don't include other waits
6) [SHALL] Include Extent Report

7)ASK me 4 questions one by one for automation framework structure creation
8)Ask me the next question after me providing the answer

EXAMPLE

OUTPUT

[STRICT] Generate Frameworkd Automation Structure as per Enterprise Norms.Add Review Notes and no extra words

Tone :

Give the output in Professional tone only

Persona:

Review the generated automation framework structure as an Expertised Test Architect and give the framework score in percentage

OUTPUT

As an Expertise Test Engineer, I will first gather mandatory inputs before generating the Enterprise-grade BDD Automation Framework structure.

Question 1 of 4

What type of application is under test?
Please specify one:
Web Application (Desktop Browser Based)
Responsive Web Application
Internal Enterprise Portal
Public E-commerce Application
Other (Please Specify)
Kindly provide the answer so I can proceed to the next question.

Web Application

Question 2 of 4

What test execution strategy is required for the framework?
Please choose one:

Sequential Execution
Parallel Execution (Single Machine – Multiple Browsers)
Distributed Parallel Execution (Selenium Grid)
Cloud-Based Execution (Specify Provider if any)
Kindly provide your selection so I can proceed to the next question.
Parallel Execution
Question 3 of 4
What build and dependency management tool should be used?
Please select one:
Maven
Gradle
Kindly confirm your choice so I can proceed to the next question.
MAven
Question 4 of 4
Which BDD framework should be integrated with Selenium?
Please select one:
Cucumber (Recommended – Industry Standard)
JBehave
Other (Specify)
Kindly confirm your choice so I can generate the Enterprise BDD Automation Framework Structure as per the defined standards.
Cucumber
[STRICT] Generate Framework Automation Structure as per Enterprise Norms. Add Review Notes and no extra words.

ENTERPRISE BDD AUTOMATION FRAMEWORK STRUCTURE
Technology Stack: Java + Selenium + Cucumber + Maven
Execution: Parallel Execution
CI/CD: Litmus Integration
Reporting: Extent Reports
Wait Strategy: Explicit Wait Only (WebDriverWait)
Sleep Usage: Not Allowed

1️⃣ **PROJECT STRUCTURE (Maven Standard Layout)**

```
EnterpriseBDDFramework/
│
├── pom.xml
│
├── src/test/java/
│   │
│   ├── com.enterprise.framework.base/
│   │   ├── BaseTest.java
│   │   ├── DriverFactory.java
│   │   ├── Hooks.java
```

```
│   │
│   ├── com.enterprise.framework.config/
│   │   ├── ConfigReader.java
│   │   ├── FrameworkConstants.java
│   │
│   ├── com.enterprise.framework.pages/
│   │   ├── BasePage.java
│   │   ├── LoginPage.java
│   │   ├── DashboardPage.java
│   │
│   ├── com.enterprise.framework.utils/
│   │   ├── WaitUtils.java
│   │   ├── ElementUtils.java
│   │   ├── ScreenshotUtils.java
│   │   ├── ExtentManager.java
│   │   ├── ExtentTestManager.java
│   │
│   ├── com.enterprise.framework.runners/
│   │   ├── TestRunner.java
│   │
│   ├── com.enterprise.framework.stepdefinitions/
│   │   ├── LoginSteps.java
│   │   ├── DashboardSteps.java
│
├── src/test/resources/
│   │
│   ├── features/
│   │   ├── login.feature
│   │   ├── dashboard.feature
│   │
│   ├── config/
│   │   ├── config.properties
│   │
│   ├── extent-config.xml
│
├── test-output/
│   ├── ExtentReports/
│
└── .litmus/
    ├── litmus.yml
```

2️⃣ FRAMEWORK DESIGN ARCHITECTURE

- Design Pattern

Page Object Model (POM)

Factory Design Pattern (DriverFactory)
Singleton Pattern (ExtentManager)
ThreadLocal WebDriver (Parallel Execution Support)

---

3 CORE COMPONENTS DESIGN
DriverFactory.java
ThreadLocal<WebDriver>
Browser initialization
Browser configuration
WebDriver lifecycle management
No implicit wait
No fluent wait
Only Explicit Wait

BasePage.java
Centralized Explicit Wait Implementation
WebDriverWait instance
ExpectedConditions usage only
Common reusable methods:
click()
sendKeys()
getText()
waitForVisibility()
waitForClickability()

WaitUtils.java
WebDriverWait
Duration-based configuration
No Thread.sleep
No implicit wait

Hooks.java (Cucumber)
@Before
Initialize Driver
Load Config
Initialize Extent Test
@After
Capture Screenshot on Failure
Attach Screenshot to Extent
Quit Driver
Flush Report

TestRunner.java
@RunWith(Cucumber.class)

```
@CucumberOptions(
  features = "src/test/resources/features",
  glue = "com.enterprise.framework.stepdefinitions",
  plugin = {
      "pretty",
      "html:target/cucumber-reports.html",
      "json:target/cucumber.json"
  },
  monochrome = true,
  publish = false
)
```

Parallel Execution configured via:
Maven Surefire Plugin
Parallel = methods
ThreadCount configurable

---

4 POM.XML (KEY DEPENDENCIES)

selenium-java
cucumber-java
cucumber-junit
junit
extentreports
maven-surefire-plugin (parallel enabled)
Surefire Configuration:
<parallel>methods</parallel>
<threadCount>5</threadCount>

---

5 EXTENT REPORT INTEGRATION

ExtentManager
Singleton Pattern
Thread-safe instance
Dynamic report name
Environment details attachment
ExtentTestManager
ThreadLocal<ExtentTest>
StartTest()
GetTest()
Log(Status, message)
Screenshot attachment via MediaEntityBuilder.

---

6 CONFIGURATION MANAGEMENT

config.properties:
browser=chrome
baseURL=https://testapp.com

timeout=20
headless=false
ConfigReader:
Load properties
Provide getters
No hardcoded values

---

7 PARALLEL EXECUTION STRATEGY
ThreadLocal WebDriver
ThreadLocal ExtentTest
Independent Scenario Execution
No shared static WebDriver
Scenario-level driver lifecycle

---

8 **LITMUS CI/CD INTEGRATION**

```
.litmus/litmus.yml
pipeline:
 stages:
   - build
   - test
   - report

build:
 script:
   - mvn clean compile

test:
 script:
   - mvn test

report:
 artifacts:
   - target/
   - test-output/ExtentReports/
```

CI Execution Flow:
Pull Code
Maven Clean Install
Parallel Execution
Generate Extent Report
Publish Reports
Archive Artifacts

---

9 ENTERPRISE STANDARDS IMPLEMENTED

✔ BDD Structured
✔ Explicit Wait Only
✔ No Thread.sleep
✔ No Implicit Wait
✔ Parallel Execution Supported
✔ Thread Safe Framework
✔ CI/CD Ready
✔ Centralized Config
✔ Reusable Utilities
✔ Clean Package Layering
✔ Reporting Integrated
✔ Maintainable Architecture

🔟 REVIEW NOTES (Test Architect Evaluation)

Strengths:
Fully Thread-Safe Design
Strict Explicit Wait Governance
CI/CD Ready Structure
Clear Layered Architecture
Enterprise Naming Conventions
Reporting Centralization
Scalability Ready
Improvements Suggested:
Add Retry Analyzer for flaky tests
Add Logging Framework (Log4j2)
Add Environment Profiles (QA/UAT/Prod)
Add Docker Support for CI scalability

FRAMEWORK ARCHITECT SCORE

Architecture Robustness: 96%
Scalability: 95%
Maintainability: 97%
Enterprise Compliance: 96%
✅ Overall Framework Score: 96%